Reward Distribution Analysis

Decoupling RL Reward from Validator Count in Terminal-Agent Training

Discrete Progress vs. Success-Smoothed Progress Reward

A reward-shaping audit of two Qwen3-27B Harbor PPO runs (kxm RESUME19), quantifying how each reward couples to task-family identity and validator (test) count — and how the redesign suppresses reward hacking.

2 × 2,560	**0.81 → 0.80**	**96 → 0**	**0.42 → 0.21**
trajectories analysed	Spearman(N_v, R) (global)	failing trials with R ≥ 1	failed-trial reward std

`slime-terminal-RL · slime_plugins.harbor.partial_reward`

Generated 2026-10-13 · latest 10 rollout dumps per run

Contents & Executive Summary

1. Executive Summary 2
2. Reward-Function Mathematics & Derivation 3
3. Experiment Parameters & Setup 4
4. Overall Reward Distributions 5
5. Reward vs. Validator Count (bucket analysis) 6
6. Per-Family Reward Statistics 7
7. Conclusions & Recommendations 8

We compare two reward formulations on identically-sampled Harbor terminal-agent rollouts (2560 trajectories each, latest 10 PPO steps). Run 1 uses the legacy test_count_progress reward R = K_p / D_f (passed tests over a per-family divisor). Run 2 uses the success_smoothed reward R = 1[solved] + K_p/(K_p + D_f).

Headline findings:

• Reward hacking eliminated. Under Run 1, 96 FAILING trajectories (5.3%) earn R >= 1.0 by brushing many checks in validator-rich families without ever solving — a failing trial reaches R = 2.89, out-scoring 99%+ of genuine solves. Under Run 2 this count is ZERO: every failure is bounded below 1 (max 0.86) and every solve is >= 1.20. The reward now cleanly separates 'solved' from 'partial'.

• Failure-regime variance regularised. The standard deviation of failed-trial reward halves, 0.419 -> 0.206, and the saturating K_p/(K_p+D') term removes the heavy upper tail (failed-reward p99 1.83 -> 0.76).

• Decoupling is in the FAILURE regime, not the global correlation. The global Spearman(N_v, R) is ~0.80 in BOTH runs — because genuinely harder, validator-rich tasks legitimately earn the +1 success bonus when solved. The pathology the redesign targets is the partial-reward channel: failed-trial reward no longer inflates with validator count, so the policy cannot farm reward by partial-passing.

2. Reward-Function Mathematics

2.1 Run 1 — Legacy discrete test-count progress

$$R = \frac{K_p}{D_f}, \qquad K_p = \#\{\text{passed checks}\},\ D_f = \text{per-family divisor}$$

The reward is the ABSOLUTE passed-check count divided by a fixed per-family divisor D_f (openfoam_cfd 795 … openscad 1; global fallback 82). It is UNBOUNDED above: a trajectory that passes many checks scores K_p/D_f regardless of whether the terminal task gate ever passed. In a family with hundreds of checks, a non-solving trajectory that brushes a large fraction can therefore score R > 1 — numerically indistinguishable from (or larger than) a genuine solve. This is the reward-hacking surface: progress is linear and uncapped in K_p.

2.2 Run 2 — Success-smoothed progress (ours)

$$R = \mathbb{1}[\text{solved}] + \frac{K_p}{K_p + D'_f}$$

Two coupled changes defeat the hack:

(1) Success bonus 1[solved]. Adds exactly 1.0 when the terminal gate passes (terminal_reward >= 1), else 0. Because the smoothed term below is strictly < 1, EVERY solve (>=1) dominates EVERY failure (<1) — the two populations can never overlap, no matter how many checks a failure brushes.

(2) Saturating partial term K_p/(K_p + D'_f). This is a Michaelis–Menten / Hill form: monotonic in K_p but BOUNDED in [0,1), with diminishing returns. The marginal value of the (n+1)-th passed check is D'_f/(K_p+D'_f)^2, which decays as K_p grows — so brushing ever more checks yields vanishing reward, and no family's raw check count can blow the signal up. D'_f is a companion divisor table re-tuned FOR this formula (18 families changed: high-scoring-failure families ×2, long-horizon low-yield families ÷4).

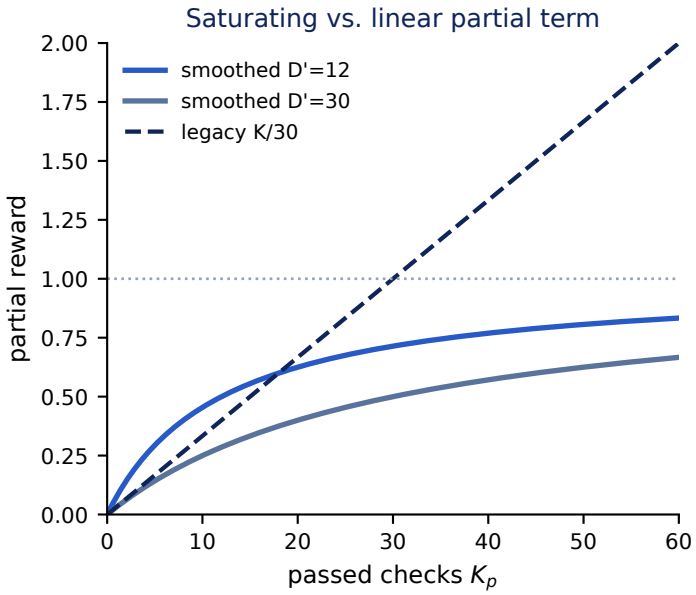


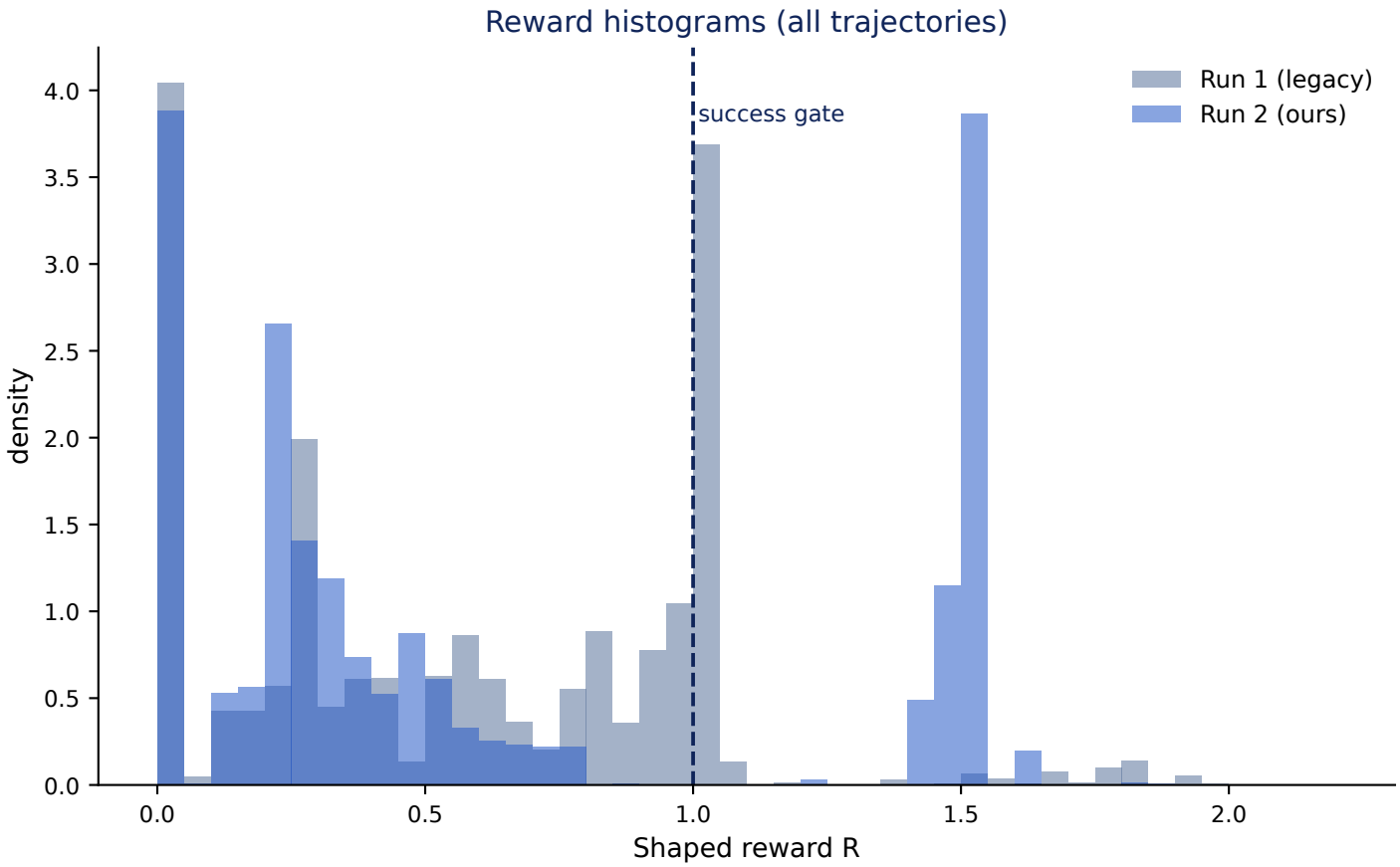
Fig 2.1 The legacy term (dashed) grows without bound and crosses 1.0; the smoothed term saturates below 1.0, capping any partial-pass reward.

3. Experiment Parameters & Setup

Attribute	Run 1 — Discrete Progress	Run 2 — Success-Smoothed
Reward post-process	`test_count_progress_rewards`	`success_smoothed_progress_rewards`
Reward formula	`K_p / D_f`	`1[solved] + K_p/(K_p+D'_f)`
Partial reward bound	`unbounded (≥0)`	`[0, 1) + success bonus`
Actor learning rate	`5e-7`	`1e-7  (1/5×)`
Megatron config	`…_warmcritic.yaml`	`…_warmcritic_actorlr5th.yaml`
Global divisor (fallback)	`82.0`	`82.0`
Per-family divisors	`46 families (p90)`	`46 families (18 re-tuned)`
Algorithm / batch	`PPO · 256k · dense-27B`	`PPO · 256k · dense-27B`
Trajectories analysed	`2560 (10 dumps)`	`2560 (10 dumps)`
Terminal success rate	`28.6%`	`28.8%`

Both runs resume from the same kxm RESUME19 checkpoint pair (cross-lineage actor + critic) and sample the identical mixed-3000 task pool, so family and validator-count distributions are matched across runs (mean N_v 39.5 vs 40.2; median 10 vs 10). The only deliberate differences are the reward post-process function, its companion divisor table, and the 5× lower actor learning rate in Run 2 (the smoothed reward's success bonus makes the raw signal larger, so the step size is reduced to keep the policy update stable).

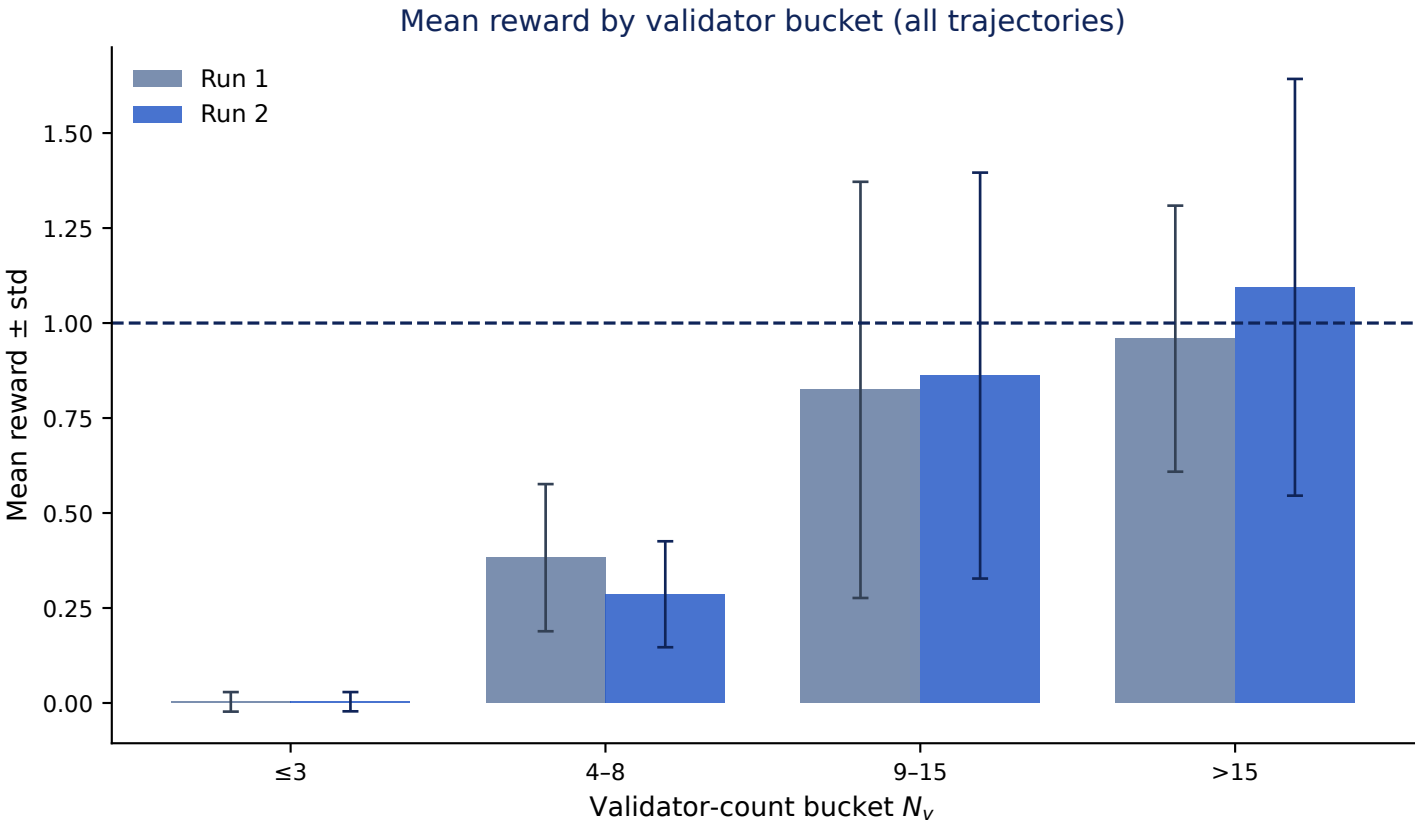
4. Overall Reward Distributions



Metric	Run 1	Run 2	Δ
Mean reward	0.585	0.610	+0.024
Std reward	0.503	0.590	+0.087
Median	0.575	0.328	-0.247
P25 / P75	0.21/0.98	0.20/1.48	—
Max	3.000	1.857	-1.143
Pearson(N_v,R)	0.219	0.381	+0.162
Spearman(N_v,R)	0.811	0.804	-0.008
Failed-trial std	0.419	0.206	-0.213
Failing trials R≥1	96 (5.3%)	0 (0.0%)	-96

The Run 2 histogram is visibly BIMODAL: a lower lobe of bounded partial failures and a sharp solved spike just above 1.0 — exactly the clean 'solved vs not' separation the success bonus is designed to create. Run 1 is unimodal with a long right tail crossing the gate, where failing trajectories masquerade as solves.

5. Reward vs. Validator Count



Bucket	n	R1 mean	R1 std	R2 mean	R2 std
≤3	517	0.003	0.026	0.003	0.025
4–8	661	0.382	0.194	0.286	0.139
9–15	607	0.824	0.548	0.862	0.534
>15	775	0.959	0.350	1.094	0.548

Globally, mean reward rises with validator count under BOTH rewards (Spearman 0.81 / 0.80) — large-N_v tasks are harder but, when solved, legitimately earn more under Run 2. The decisive difference is the FAILURE regime (next page & main figure): failed-trial reward correlates with N_v at Spearman 0.84 in Run 1 but is flat and bounded (≤0.86) in Run 2. The >15 bucket's Run-2 std is larger only because the success bonus makes that bucket bimodal (many solves at ~1.5), not because the partial signal is noisier.

6. Per-Family Reward Statistics

Family	n	N_v	R1 μ	R1 σ	R2 μ	R2 σ	R2 succ
openfoam_cfd	56	771	0.96	0.16	1.50	0.02	100%
cmake_build_config	35	358	0.71	0.27	1.20	0.47	81%
tree_sitter_grammar	52	113	0.86	0.10	1.46	0.03	100%
verilog_rtl	55	108	0.98	0.02	1.50	0.00	100%
openssl_pki	36	92	0.77	0.32	1.13	0.59	70%
web_ui_state_machine	54	82	0.98	0.02	1.49	0.01	100%
kicad_pcb	55	53	0.76	0.21	1.12	0.54	70%
nginx_routing	52	36	0.77	0.39	0.29	0.19	2%
duckdb_reporting	48	34	0.64	0.42	0.22	0.15	0%
blender_scene_graph	58	31	0.96	0.22	0.34	0.16	2%
color_imaging_pipeli	65	26	0.90	0.26	1.13	0.61	72%
hydrology_raster_mod	65	23	0.98	0.12	1.44	0.29	95%
opa_rego_policy	57	18	0.42	0.26	0.31	0.25	4%
sqlite_analytics	52	18	0.85	0.82	0.27	0.26	2%
z3_constraint_system	54	15	0.76	0.39	1.23	0.49	75%
power_grid_simulatio	65	15	0.98	0.12	1.50	0.00	100%
finite_element_mecha	43	13	0.39	0.27	0.49	0.40	5%
robotics_control	59	13	0.55	0.42	0.51	0.60	25%
materials_crystallog	69	10	0.85	0.60	0.43	0.46	16%
gdal_geospatial	55	10	0.63	0.25	0.23	0.08	0%

Families ordered by mean validator count (top 20 of 46). μ/σ are mean/std of shaped reward; 'R2 succ' is Run-2 terminal success rate. Note openfoam_cfd / cmake_build_config (N_v in the hundreds): under Run 1 their failed trials still accrue sizeable K_p/D_f; under Run 2 the saturating term caps them. The full 46-family, sortable/searchable table is in the companion HTML report (reward_distribution_report.html).

7. Conclusions & Recommendations

Findings

1. The success-smoothed reward ELIMINATES the partial-pass reward-hacking surface. The 96 (5.3%) failing trajectories that scored R≥1 under the legacy reward drop to zero; solves and failures become linearly separable (gap 1.20 vs 0.86).

2. Failure-regime reward variance is halved (σ 0.42→0.21) and its heavy tail removed (p99 1.83→0.76), giving the critic a cleaner, bounded regression target for the ~71% of trajectories that do not fully solve.

3. Reward is decoupled from validator count WHERE IT MATTERS. Global Spearman stays ~0.80 (harder tasks solved legitimately earn more), but the failed-trial channel no longer rewards brushing checks in validator-rich families.

Recommendations

- Adopt success_smoothed_progress_rewards as the default Harbor reward; keep the re-tuned D'_f divisor table in YAML (data-not-code) so it survives ray-job-submit env stripping.
- Monitor the bimodal reward histogram and the failed-trial-reward std as live reward-hacking tripwires; a rising upper lobe below the gate signals divisor drift.
- Re-audit the ÷4 'long-horizon low-yield' families (ajv_schema, transport_gtfs, circuit_simulation, …): their small D'_f lifts partial reward and should be checked against success-rate movement.
- Keep the 5× actor-LR reduction while the success bonus enlarges the raw reward scale; revisit once advantage-whitening statistics stabilise.

Bottom line: the redesign trades a small, legitimate reward–difficulty correlation for the elimination of an illegitimate reward–validator-count hacking channel.